



Lab Report – EE-170L Computer Programming

NAME	Danyal Hassan
ID	750
SEMESTER	2nd
LAB	Programming
DATE	26/06/2026

COMPUTER PROGRAMMING

Electrical Engineering Department — UET Nowshera

LAB 10

Lab Title: More About Functions in C++

1. OBJECTIVE

The objectives of this lab are:

- To understand the difference between call by value and call by reference in C++.
- To observe how call by value prevents modification of the original variable.
- To observe how call by reference allows a function to modify the caller's variable.
- To learn how to specify and use default parameter values in function declarations.

2. THEORY

Call by Value

Call by value is used when the called function does not need to modify the original variable in the caller. A copy of the variable is passed to the function, so any changes the function makes to its local copy have no effect on the original variable in main. This is the default method of passing arguments in C++.

Example — modifying a copy inside a function:

```
#include <iostream>
using namespace std;

void func(int); // function prototype
```

```

int main()
{
    int i = 10;    func(i); // passes a copy of i
    cout << "The value of i remains " << i << endl;
    return 0;
}

// function definition: only the local copy is changed
void func(int i)
{
    i = i + 10; // modifies the local copy, not the original
    cout << "Inside function, the value of i changed to " << i << endl;
}

```

Output: Inside function, the value of i changed to 20 | The value of i remains 10

Call by Reference

Call by reference passes the memory address of the original variable to the function using the & (ampersand) operator. The function works directly on the original variable rather than a copy, so any changes made inside the function are reflected in the caller. This is useful when a function needs to modify the caller's data or return more than one value.

Example — swapping two values using call by reference:

```

#include <iostream>
using namespace std;

void swapping(int &, int &); // prototype: reference parameters

int main()
{
    int x = 50, y = 70;    cout << "Before: x = " << x
    << ", y = " << y << endl;    swapping(x, y); // passes
    addresses of x and y    cout << "After:  x = " << x <<
    ", y = " << y << endl;    return 0;
}

// function definition: swaps the original variables
void swapping(int &x1, int &y1)
{
    int z1;    z1 = x1; //
    temporary store

    x1 = y1; // overwrite x1 with y1
    y1 = z1; // complete the swap }

```

Output: Before: x = 50, y = 70 | After: x = 70, y = 50

Default Values in Parameters

When declaring a function, a default value can be assigned to any of the trailing parameters. If the caller omits that argument, the default value is used automatically. If the caller provides a value, the default is ignored and the provided value is used instead.

Example — function with default parameter values:

```
#include <iostream>
using namespace std;

// default values assigned in the definition
int divide(int a = 8, int b = 2)
{
    int r;    r = a / b; //
integer division    return r;
}

int main()
{
    cout << divide()    << endl; // uses defaults: 8/2 = 4    cout
    << divide(12)    << endl; // uses a=12, default b=2: 12/2 = 6    cout
    << divide(63,7) << endl; // both provided: 63/7 = 9    return 0;
}
```

Output: 4 | 6 | 9

3. SOFTWARE USED

- DEV-C++ IDE
- C++ Compiler (bundled with DEV-C++)

4. TASKS

Task 1 — Function with Parameters

A

```
#include <iostream>
using namespace std;

int getSquare(int number); // prototype: returns an integer

int main()
```

function named multiply was declared with two integer parameters, num1 and num2. Inside the function the product of the two numbers is calculated and displayed. The function is called from main with two integer arguments.

```
#include <iostream>
using namespace std;

void multiply(int num1, int num2); // prototype: two integer parameters

int main()
{
    multiply(6, 7); // call with two arguments
    return 0;
}

// function definition: computes and displays the product
void multiply(int num1, int num2)
{
    int product = num1 * num2; // calculate product
    cout << "Product = " << product << endl; // display result
}
```

Task 2 — Function with Return Value

A function named getSquare was declared with one integer parameter, number. Inside the function the square of the number is computed and returned to the caller. The main function calls getSquare, receives the returned value, and displays it.

```

{
    int num;      cout << "Enter
a number: ";    cin >> num;

    int result = getSquare(num); // call and store the returned
value      cout << "Square = " << result << endl;    return 0;
}

// function definition: calculates and returns the square
int getSquare(int number)
{
    return number * number; // return square to the caller
}

```

5. CONCLUSION

This lab explored three important aspects of C++ functions: call by value, call by reference, and default parameter values. Call by value was shown to protect the original variable from modification, while call by reference was demonstrated as an effective way to allow a function to modify caller variables and effectively return multiple results. Default parameter values were used to make functions more flexible by allowing some arguments to be omitted at the call site. These concepts are essential tools for writing clean and efficient C++ programs.